FORMAN CHRISTIAN COLLEGE

(A CHARTERED UNIVERSITY)

CSCS 306 - A

FALL 24

Lab 3 Report

Hafsah Shahbaz – 251684784

Daim Bin Khalid - 251686775

Syeda Manal Ammad - 251606966

8th October 6, 2024

Introduction

In this lab, we develop an Arduino-based program to simulate a fuel dispensing system using a 4x3 matrix keypad and an LED to provide user feedback. The system prompts the user to input an amount via the keypad, calculates the equivalent fuel that can be dispensed based on the input, and simulates the fuel dispensing process by controlling the LED. The program demonstrates several important functionalities, including handling keypad input, converting the input into a usable format, and controlling external hardware components like LEDs.

Functions and Libraries

Keypad Library: This library enables easy interaction with the 4x3 matrix keypad. The keypad allows users to input numeric values that represent the fuel amount.

Functions

- **keypad.getKey(): Detects the key press from the keypad, identifying whether it is a numeric value (0-9) or a control key ('*' for clearing input and '#' for submitting the entered amount).**
- **String inputAmount: Stores the user-entered numeric values as a string, which is then converted into an integer for further processing.**
- **inputAmount.toInt(): Converts the string input into an integer, which represents the fuel amount to be dispensed.**
- **digitalWrite(): Controls the LED, turning it on and off to simulate fuel being dispensed based on the user's input.**

Algorithm and Logic

1. **Setup Phase**:
    - Initialize serial communication.
    - Set the LED pin to output mode.
    - Provide feedback to the user by blinking the LED three times to indicate system readiness.
    - Display the message prompting the user to enter the fuel amount.
2. **Setup Phase**:
    - Continuously check the keypad for input using `keypad.getKey()`.
    - If a numeric key (0-9) is pressed, append it to the `inputAmount` string, which stores the user's input.
    - Check for 2- 4-digit range.
    - If the '#' key is pressed, proceed to the conversion phase.

- If the '*' key is pressed, clear the input and reset `inputAmount` to an empty string.

3. **Conversion Phase**:
   - Convert the `inputAmount` string to an integer using `inputAmount.toInt()` to represent the fuel amount.
   - Display the entered amount on the Serial Monitor.
   - Calculate the number of litres to dispense based on the fuel amount by dividing it by 200.
4. **Validation and Dispensation Phase**:
   - Check if the calculated litres are greater than or equal to 1:
     - If true, display the number of litres to be dispensed, blink the LED for each unit of fuel dispensed, and display a message indicating the completion of the dispensing process.
     - If false, display a message that the amount entered is too small to dispense any fuel.
   - If the '*' key is pressed, the system clears the input, resets inputAmount, and displays a message indicating that the input has been cleared.
5. **Output Phase**:
   - Provide feedback on the Serial Monitor, displaying the entered fuel amount and the calculated number of litres.
   - If fuel was dispensed, indicate that the process is complete and prompt the user with a "Drive safe" message.
   - If no fuel was dispensed, indicate that the entered amount was too low for dispensing.

Code Breakdown

```
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

int LED = 2;

String inputAmount = "";

int fuelAmount = 0;
```

Function Declarations: Declare all necessary variables such as the keypad object, LED pin, a string for storing the fuel amount (inputAmount), and an integer for the fuel amount (fuelAmount).

```
void setup() {
```

```
    Serial.begin(9600);
    pinMode(LED, OUTPUT);
    for (int i = 0; i < 3; i++) {
        digitalWrite(LED, HIGH);
        delay(500);
        digitalWrite(LED, LOW);
        delay(500);
    }
    Serial.println("Enter amount:");
}
```

Setup Function: Initializes the Serial Monitor for communication and sets the LED pin as output. It also provides feedback by blinking the LED three times to show that the system is ready for input.

```
void loop() {
    char key = keypad.getKey();
    if (key) {
        if (key >= '0' && key <= '9') {
            inputAmount += key;
            Serial.print(key);
        } else if (key == '#') {
            fuelAmount = inputAmount.toInt();
            ...
        }
    }
}
```

loop(): Continuously checks for input from the keypad. If a number is pressed, it appends the value to the inputAmount string and displays it on the Serial Monitor. When the # key is pressed, it triggers the process of converting the input to an integer (fuel amount) and proceeds with further calculations.

```
inputAmount.toInt();
```

inputAmount.toInt(): Converts the user-entered numeric string (inputAmount) into an integer for further calculations, specifically to determine the fuel amount to be dispensed.

```
digitalWrite(LED, HIGH);
digitalWrite(LED, LOW);
```

digitalWrite(): Controls the LED to simulate fuel dispensing. It turns the LED on and off, where each blink represents the dispensing of one unit of fuel based on the input amount.

```
if (fuelAmount > 0) {
    int liters = fuelAmount / 200;
    if (liters >= 1) {
        for (int i = 0; i < liters; i++) {
            digitalWrite(LED, HIGH);
            delay(1000);
            digitalWrite(LED, LOW);
            delay(1000);
        }
    }
}
```

Fuel Dispensing Logic: Checks if the entered fuel amount is valid (greater than 0). It calculates the number of liters by dividing the fuel amount by 200. If at least one liter can be dispensed, the program blinks the LED once per liter to simulate the dispensing process.

```
if (key == '*') {
```

```
    inputAmount = "";
    Serial.println("Input cleared.");
}

Serial.println("Thank you for visiting us. \nDrive safe.");
```

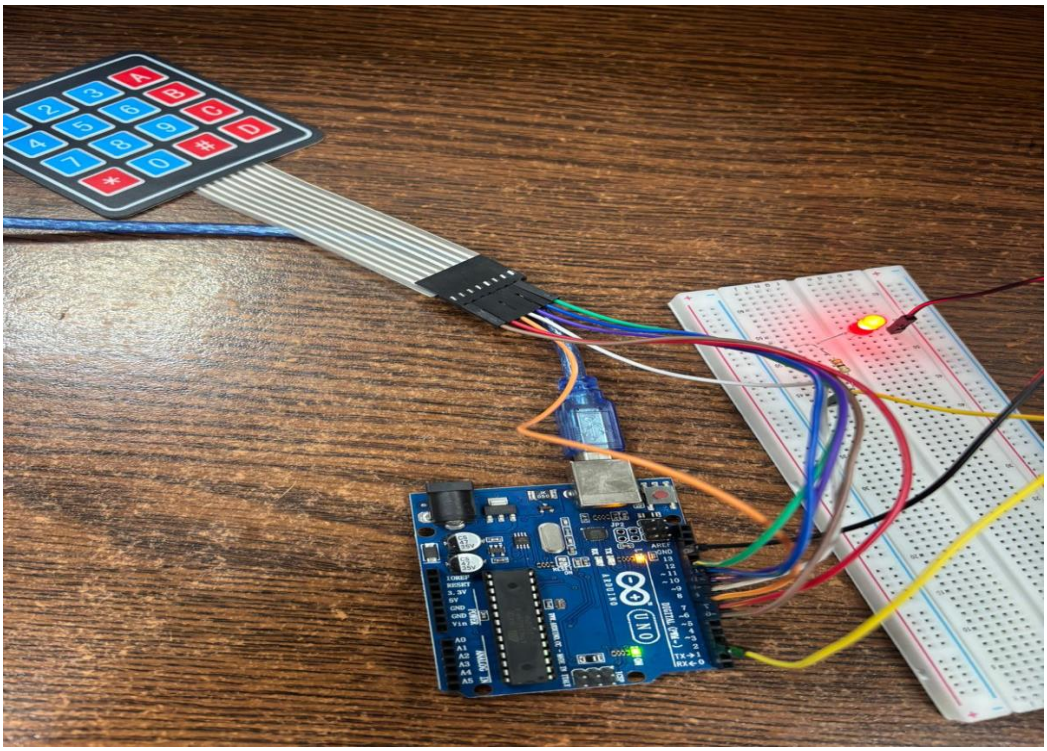
Clearing Input: If the * key is pressed, the system resets the input by clearing the inputAmount string and provides feedback to the user that the input has been cleared.

Final Message: Once the fuel is dispensed or if the fuel amount is too small, a final message is printed to the Serial Monitor, thanking the user and prompting them to drive safely.

Output

```
COM7

Enter amount:
170
You entered: 170
Sorry, system cannot dispense fuel against this amount.
Thank you for visiting us.
Drive safe.
```



Comment: The output demonstrates that the code successfully simulates a fuel dispenser mechanism, and the user interaction is successfully implemented through the keypad using serial monitor as display.